

# JavaScript Bitwise Operators & Bit Manipulation — Interview Cheat Sheet

Made for quick recall during prep. All examples are in JavaScript (32-bit integer bitwise semantics).

## 1) Bitwise Operators (JS)

| Operator       | Symbol | Meaning                          | Example            | Result     |
|----------------|--------|----------------------------------|--------------------|------------|
| AND            | &      | 1 if both bits are 1             | 5 & 3 // 101 & 011 | 1          |
| OR             |        | 1 if either bit is 1             | 5   3 // 101   011 | 7          |
| XOR            | ^      | 1 if bits differ                 | 5 ^ 3 // 101 ^ 011 | 6          |
| NOT            | ~      | Flip all bits (two's complement) | ~5                 | -6         |
| Left Shift     | <<     | Shift left i ( $\times 2^i$ )    | 5 << 1             | 10         |
| Right Shift    | >>     | Arithmetic right (sign-prop)     | -5 >> 1            | -3         |
| Unsigned Right | >>>    | Logical right (zero-fill)        | -5 >>> 1           | 2147483645 |

### Notes:

- Bitwise ops use 32-bit signed integers internally. Non-integers are converted via `ToInt32`. Use `>>> 0` to coerce to unsigned 32-bit.
- Beware of overflow on large numbers; prefer `BigInt` + custom ops if needed.

## 2) Common Helpers (Set / Clear / Toggle / Check / Count)

```
function isBitSet(n, i) { return (n & (1 << i)) !== 0; }
function setBit(n, i) { return n | (1 << i); }
function clearBit(n, i) { return n & ~(1 << i); }
function toggleBit(n, i) { return n ^ (1 << i); }
function countBits(n) { let c=0; while(n){ n&=(n-1); c++; } return c; } // Brian Kernighan
```

### Examples:

```
isBitSet(5,0) → true    // 5=101
setBit(5,1) → 7          // 111
clearBit(5,0) → 4        // 100
toggleBit(5,2) → 1       // 001
countBits(13) → 3       // 1101
```

## 3) XOR Tricks

Key identities:  $a \oplus a = 0$ ,  $a \oplus 0 = a$ ,  $a \oplus b \oplus b = a$ .

```
// Single number (others twice) function singleNumber(arr){ return
arr.reduce((x,n)=>x^n,0); }
singleNumber([2,3,2,4,4]) → 3
```

```
// Swap without temp let a=5,b=7; a^=b; b^=a; a^=b; // a=7,b=5
```

## 4) Bit Masking & Subsets

Use an integer to represent a set. If bit *i* is 1, element *i* is in the set.

```
const items=["a","b","c"];
for(let mask=0; mask<(1<< items.length); mask++){
  const subset=[];
  for(let i=0;i<items.length;i++){
    if((mask & (1<< i)) > 0) subset.push(items[i]);
  }
}
```

```
// Iterate all submasks of a mask
function forEachSubmask(mask, fn){
  for(let sub=mask; sub; sub=(sub-1)&mask;) fn(sub);
}
forEachSubmask(0b10110, s=>console.log(s.toString(2)));
```

## 5) State Compression DP (Assignment Problem)

$dp[mask]$  = minimum cost to assign the first  $popcount(mask)$  workers to the set of jobs in 'mask'.

```
function countBits(n){ let c=0; while(n){ n&=(n-1); c++; } return c; }
function assignmentDP(cost){
  const n=cost.length, N=1<<n;
  for(let mask=0; mask<N; mask++) const i=countBits(mask);
  for(let j=0; j<n; j++) const m=mask|(1<<j);
  dp[m]=Math.min(dp[m], dp[mask]+cost[i][j]);
}
return dp[N-1];
}
// Example const cost=[[9,2,7],[6,4,3],[5,8,1]]; // → 9
```

## 6) Pitfalls & Tips (JS-specific)

- Numbers are IEEE754 doubles; bitwise ops coerce to 32bit signed ints.
- Use  $(x \ggg 0)$  to coerce to unsigned 32bit when you want zero-fill semantics.
- Prefer `countBits` Kernighan's loop over converting to binary strings.
- For negative numbers, remember  $\gg$  is arithmetic,  $\ggg$  is logical (fills with zeros).

## 7) Practice Checklist

- Single number / two unique numbers using XOR
- Detect power of two:  $(n \& (n-1)) === 0$  ( $n > 0$ )
- Reverse bits, count bits, parity
- All subsets with bitmask; iterate all submasks
- DP on subsets: assignment / TSP (small N)